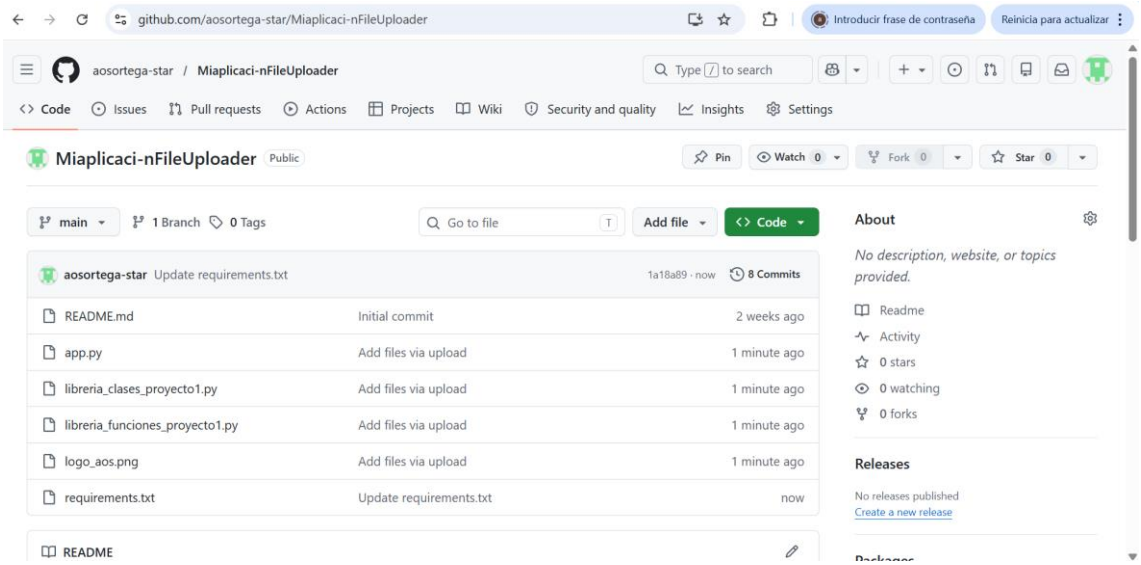
Nombre:Alex Arturo Ortega Soto

Fecha: 02/05/2026

Nombre curso: Esp. Python ed.57

Título del Proyecto: Guia Trabajo Práctico Módulo Python Fundamentals



https://github.com/aosortega-star/Miaplicaci-nFileUploader

Comentarios_.

El curso de Python es super interesante y es necesario seguir profundizando e investigando

He logrado los objetivos propuestos

Codigo del proyecto:

```
import streamlit as st

import pandas as pd

import numpy as np

import math

import libreria_funciones_proyecto1 as lfp

import libreria_clases_proyecto1 as lcp
```

```
# Inicializando las bases de datos en `st.session_state` si no existen.

# `st.session_state` se usa para almacenar variables en memoria durante una sesión en Streamlit.

columnas=["id", "nombre","salario_base","bono", "descuento","salario_neto"]

if 'df_empleado4' not in st.session_state:

    st.session_state.df_empleado4 = pd.DataFrame(columns=columnas)


if 'movimientos' not in st.session_state:

    st.session_state['movimientos'] = []  # Diccionario vacío para almacenar los movimientos


if 'arr_productos' not in st.session_state:

    st.session_state.arr_productos = np.array([])

    st.session_state.arr_cantproducto = np.array([])

    st.session_state.arr_precio = np.array([])

    st.session_state.arr_cantidad = np.array([])

    st.session_state.arr_total = np.array([])


if 'arr_monto' not in st.session_state:

    st.session_state.arr_monto = np.array([])

    st.session_state.arr_tasa = np.array([])

    st.session_state.arr_plazo_m = np.array([])

    st.session_state.arr_cuota_m = np.array([])

    st.session_state.arr_prestamo = np.array([])

    st.session_state.arr_ingreso = np.array([])


if "mi_texto" not in st.session_state:

    st.session_state.mi_texto = ""


if "mi_numinput" not in st.session_state:

    st.session_state.mi_numinput = 0.0
```

```
def limpiar_texto():
    st.session_state.mi_texto = ""
    st.session_state.mi_numinput=0.0




def agregar_movimientos(concepto, tipo_movimiento, ingreso_real, gasto_real):
    """
    Esta función recibe los datos de movimientos y lo agrega a la base de datos en memoria.
    """
    nuevo_movimiento = {
        'concepto': concepto,
        'tipo_movimiento': tipo_movimiento,
        'ingreso_real': ingreso_real,
        'gasto_real': gasto_real
    }
    if valor > 0:
    #agregar_movimientos(concepto, tipo_movimiento, Ingreso_r, Gasto_r)
        st.session_state['movimientos'].append(nuevo_movimiento)
        st.success(f"Movimiento '{concepto}' agregado.")
        return True
    else:
        st.error("Monto ingresado no puede ser cero")
        return False

# Funciones para mostrar los DataFrames
def mostrar_movimientos():
    if len(st.session_state['movimientos']) > 0:
        # Pandas convierte automáticamente una lista de diccionarios en un DataFrame
        df = pd.DataFrame(st.session_state['movimientos'])
```

```
        st.subheader("Listado de Movimientos")
        st.dataframe(df, width='stretch') # st.dataframe se ve más ordenado que st.write
        total_i = df["ingreso_real"].sum()
        total_g = df["gasto_real"].sum()
        total_saldo =  round(float(total_i - total_g),2)

        st.write(":blue[Total Ingresos:]", total_i)
        st.write(":red[Total Gastos:]", total_g)
        st.write("**Total Saldo:**", total_saldo)
        #st.metric(label="Total Saldo A:", value=f"{total_saldo}", delta= total_i - total_g,
delta_color="normal")
        #st.metric(label="Total Saldo A:", value=f"{total_saldo}", delta= total_i - total_g,
delta_color="normal")
        if total_saldo > 0:
            mensaje_s =" positivo"
            st.write(f":blue[El saldo de balance es: {mensaje_s}]")
        else:
            mensaje_s =" negativo"
            st.write(f":red[El saldo de balance es: {mensaje_s}]")
    else:
        st.info("Aún no hay movimientos registrados.")

def agregar_productos(producto, cat_producto, precio, cantidad, total):
    """
    Esta función recibe los datos de movimientos y lo agrega a la base de datos en memoria.
    """
    #nuevo_producto = {
    #    'producto': producto,
    #    'cat_producto': cat_producto,
    #    'precio': precio,
    #    'cantidad': cantidad,
```

```
    #    'total': total,
    #}
    if cantidad > 0:
        st.session_state.arr_productos = np.append(st.session_state.arr_productos, producto)
        st.session_state.arr_cantproducto = np.append(st.session_state.arr_cantproducto,
cat_producto)
        st.session_state.arr_precio = np.append(st.session_state.arr_precio, precio)
        st.session_state.arr_cantidad = np.append(st.session_state.arr_cantidad, cantidad)
        st.session_state.arr_total = np.append(st.session_state.arr_total, total)
        st.success(f"Producto '{producto}' agregado.")
        return True
    else:
        st.error("producto no ingresado")
        return False

def mostrar_productos():
    if len(st.session_state['arr_productos']) > 0:
        # Pandas convierte automáticamente una lista de diccionarios en un DataFrame
        df = pd.DataFrame({"producto:": st.session_state.arr_productos,
            "categoria producto": st.session_state.arr_cantproducto,
            "precio": st.session_state.arr_precio,
            "cantidad": st.session_state.arr_cantidad,
            "total": st.session_state.arr_total}
        )

        st.subheader("Listado de Productos")
        st.dataframe(df, width='stretch') # st.dataframe se ve más ordenado que st.write

    else:
        st.info("Aún no hay productos registrados.")
```

```
def agregar_cuota(monto, tasa_a, plazo_m, cuota_m, total_p, interes_t):
    """
    Esta función recibe los datos de prestamo y los registra
    """
    #nuevo_prestamo = {
    #    'Monto': monto,
    #    'Tasa Anual': tasa_a,
    #    'Plazo Meses': plazo_m,
    #    'Cuota Mensual': cuota_m,
    #    'Total Prestamo': total_p,
    #    'Interes Total': interes_t
    #}
    if monto > 0:
        st.session_state.arr_monto = np.append(st.session_state.arr_monto, monto)
        st.session_state.arr_tasa = np.append(st.session_state.arr_tasa, tasa_a)
        st.session_state.arr_plazo_m = np.append(st.session_state.arr_plazo_m, plazo_m)
        st.session_state.arr_cuota_m = np.append(st.session_state.arr_cuota_m, cuota_m)
        st.session_state.arr_prestamo = np.append(st.session_state.arr_prestamo, total_p)
        st.session_state.arr_interes = np.append(st.session_state.arr_ingreso, interes_t)
        st.success(f"Prestamo '{monto}' agregado.")
        return True
    else:
        st.error("Prestamo no ingresado")
        return False

def mostrar_prestamo():
    if len(st.session_state['arr_monto']) > 0:
        # Pandas convierte automáticamente una lista de diccionarios en un DataFrame
        df = pd.DataFrame({"Monto:": st.session_state.arr_monto,
            "Tasa Anual": st.session_state.arr_tasa,
            "Plazo en Meses": st.session_state.arr_plazo_m,
```

```
            "Cuota Mensual": st.session_state.arr_cuota_m,
            "Prestamo": st.session_state.arr_prestamo,
            "Interes Total": st.session_state.arr_interes
            }
        )

        st.subheader("Listado de Prestamo")
        st.dataframe(df, width='stretch') # st.dataframe se ve más ordenado que st.write

    else:
        st.info("Aún no hay prestamos registrados.")

class EMPLEADOCRUD:
    def __init__(self, df):
        self.df = df

    def agregar_registro(self, registro):
        nuevo_df = pd.DataFrame([registro])
        self.df = pd.concat([self.df, nuevo_df], ignore_index=True)

    def obtener_registro(self, id):
        return self.df[self.df["id"] == id]

    def actualizar_registro(self, id, columna, valor):
        self.df.loc[self.df["id"] == id, columna] = valor

    def eliminar_registro(self, id):
        self.df = self.df[self.df["id"] != id]

    def mostrar(self):
        return self.df
```

```
# Inicialización de estado en Streamlit

st.title("📊 Proyecto - Streamlit")


# Menú de navegación

opcion = st.sidebar.selectbox("Selecciona una acción", ["Home", "Ejercicio1", "Ejercicio2",
"Ejercicio3","Ejercicio4"])


# Agregar nueva actividad

if opcion == "Home":

    #st.subheader("➕ Agregar Nueva Actividad")

    titulo = "Módulo 1 – Python Fundamentals"


    st.image("logo_aos.png", width =150)

    st.subheader(titulo)

    st.markdown("**Nombre: Alex Arturo Ortega Soto**")

    st.write("2026")

    st.markdown(":blue[Este proyecto busca mostrar los conceptos aprendidos durante el primer
modulo de Python Fundamentals]")

    st.markdown("""

            **Tecnologias usadas**:

            - Python: (Pandas, Numpy, Streamlit)

            - Git

            - Visual Studio Code

            - Colab

            """)

elif opcion == "Ejercicio1":

    st.markdown(":blue[Ejercicio 1]")

    st.markdown("""

            **Aplicar y demostrar el uso de dataframe y captura de datos

              usando controles como:**:

            - Selectbox
```

```
            - number_input
            - text_input
            """)
    concepto = st.text_input("Concepto",key="mi_texto")
    tipo_movimiento = st.selectbox("Tipo Movimiento", ["Ingreso","Gasto"],key=
'tipo_movimiento')
    tipo_mov = str(tipo_movimiento)
    valor = st.number_input(tipo_mov + " Real", min_value=0.0, key="mi_numinput")
    if tipo_mov == "Ingreso":
        Ingreso_r = valor
        Gasto_r = 0
    elif tipo_mov == "Gasto":
        Gasto_r = valor
        Ingreso_r = 0

    if st.button("Agregar Movimiento"):
        agregar_movimientos(concepto, tipo_mov, Ingreso_r, Gasto_r)
        mostrar_movimientos()

elif opcion == "Ejercicio2":
    st.markdown(":blue[Ejercicio 2]")
    st.markdown("""
            **Aplicar y demostrar el uso de librerias
              como NumPy y manejo de Arrays y Dataframe:**:
            """)
    e2_producto = st.text_input("Nombre de Producto")
    e2_catproducto = st.selectbox("Categoria",
["Alimentos","Tecnologia","Bebidas","Ropa"],key= 'tipo_categoria')
    e2_precio = st.number_input("Precio: ", min_value=0.0, key="mi_precio")
    e2_cantidad = st.number_input("Cantidad: ", min_value=0, key="mi_cantidad")
    e2_total = e2_precio * e2_cantidad
    #st.write("**Total Saldo:**", total_saldo)
```

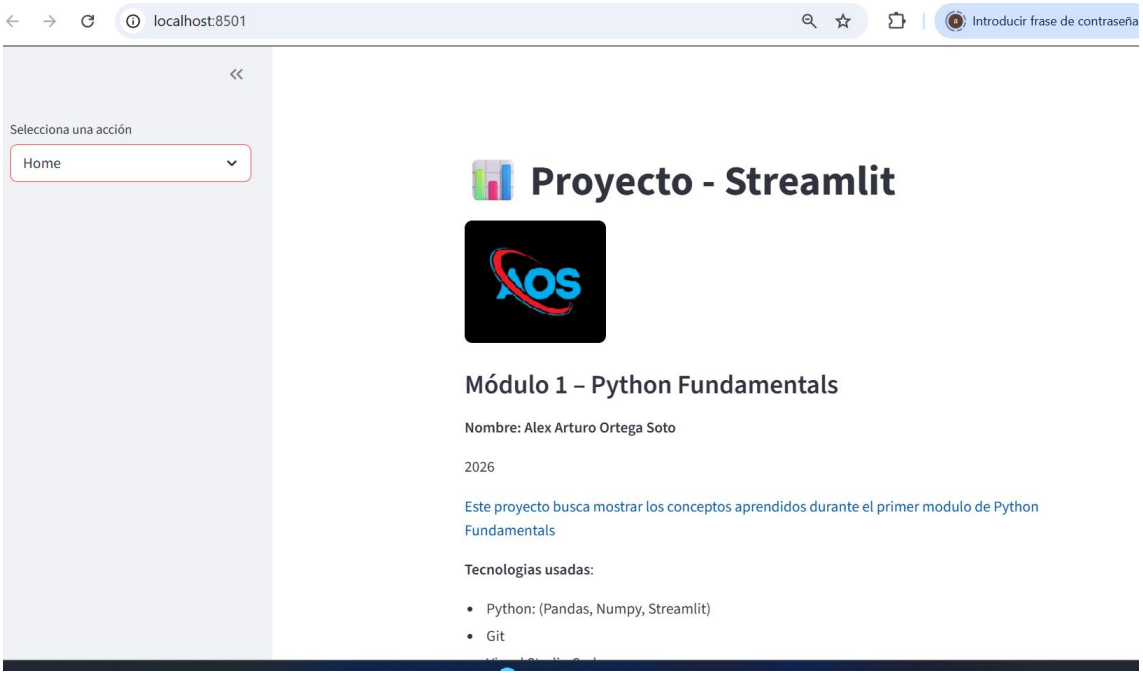
```
    st.metric(label="Total: ", value=f"{e2_total}")
    if st.button("Agregar Productos"):
        agregar_productos(e2_producto, e2_catproducto, e2_precio, e2_cantidad, e2_total)
        mostrar_productos()
elif opcion == "Ejercicio3":
    st.markdown(":blue[Ejercicio 3]")
    st.markdown("""
            **Uso de funcion calcular_cuota_prestamo_frances
              de la libreria de funciones:**:
            """)


    e3_monto = st.number_input("Monto de Prestamo", min_value=0)
    e3_tasa_anual = st.number_input("Tasa Anual: ", min_value=0.00, max_value=100.00)
    e3_plazo_meses = st.number_input("Plazo Meses: ", min_value=1, max_value=240)
    #st.write("**Total Saldo:**", total_saldo)
    if st.button("Calcular Cuota"):
        #e3r_cuota_mensual, e3r_total_pagado, e3r_interes_total =
lfp.calcular_cuota_prestamo_frances(e3_monto, e3_tasa_anual, e3_plazo_meses)
        e3r_resultados = lfp.calcular_cuota_prestamo_frances(e3_monto, e3_tasa_anual,
e3_plazo_meses)
        st.write("cuota_mensual",e3r_resultados["cuota_mensual"])
        st.write("total_pagado",e3r_resultados["total_pagado"])
        st.write("interes_total",e3r_resultados["interes_total"])
        #st.write("e3r_total_pagado", e3r_resultados[2])
        #st.write("e3r_interes_total", e3r_resultados[3])
        agregar_cuota(e3_monto, e3_tasa_anual,
e3_plazo_meses,e3r_resultados["cuota_mensual"]
,e3r_resultados["total_pagado"],e3r_resultados["interes_total"])
        mostrar_prestamo()
elif opcion == "Ejercicio4":
    st.markdown(":blue[Ejercicio 4]")
    st.markdown("""
```

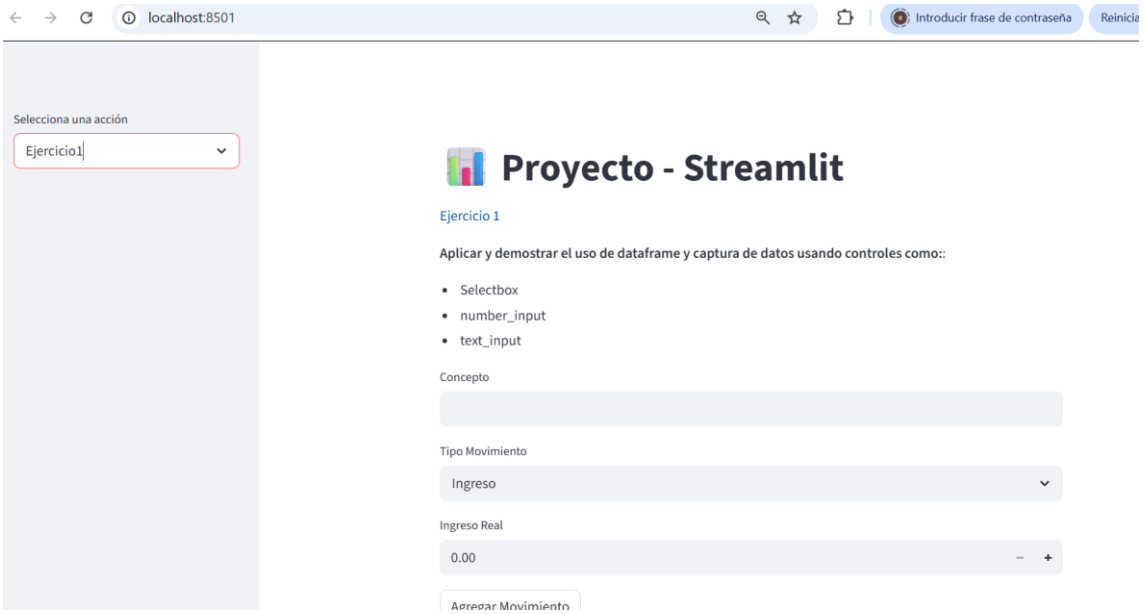
```
            **Uso de funciones de
              de CRUD:**:
            """)
    e4_nombre = st.text_input("Nombre Empleado", key="e4_nombre_key")
    #e4_catproducto = st.selectbox("Categoria",
["Alimentos","Tecnologia","Bebidas","Ropa"],key= 'tipo_categoria')
    e4_salario = st.number_input("Salario", min_value=0,key="e4_salario_key")
    e4_bono = st.number_input("Porcentaje Bono: ", min_value=0.00, max_value=100.00)
    e4_descuento = st.number_input("Porcentaje Descuento: ", min_value=0.00,
max_value=100.00)


    crud_empleado = EMPLEADOCRUD(st.session_state.df_empleado4)
    dt_emp  = crud_empleado.mostrar()
    event = dt_emp
    st.subheader("Selecciona una acción")
    col1, col2, col3, col4 = st.columns(4)
    with col1:
      if st.button("Registrar"):
        st.write("INGRESO")
        Empleado = lcp.Empleado(e4_nombre, e4_salario, e4_bono, e4_descuento )
        datos_empleado = Empleado.resumen()
        indice = crud_empleado.df["id"].max()
        if pd.isna(indice):
          indice = 0
        else:
          indice = indice +1
        st.write("valor de id: ", indice)
        nuevo_empleado = {"id":indice, #crud_empleado.df["id"].max() + 1,
        "nombre": datos_empleado["nombre"],
        "salario_base": datos_empleado["salario_base"],
        "bono": datos_empleado["bono"],
        "descuento": datos_empleado["descuento"],
```

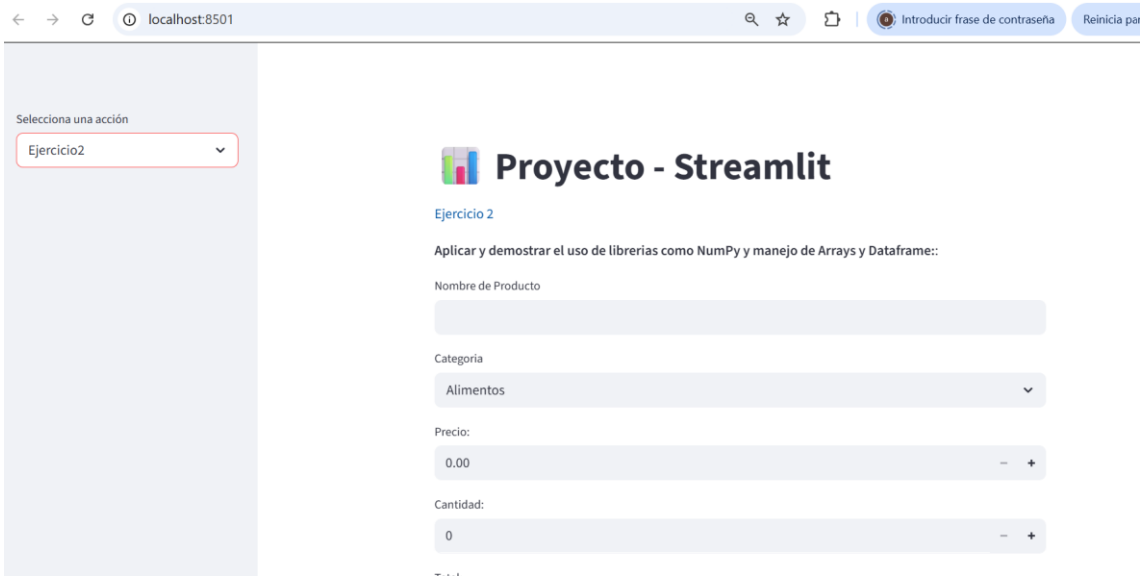
```
        "salario_neto": datos_empleado["salario_neto"]
        }

        crud_empleado.agregar_registro(nuevo_empleado)
        st.session_state.df_empleado4 = crud_empleado.df
        st.rerun()
        #dt_emp  = crud_empleado.mostrar()
        st.write("Selecciona una fila para modificar o eliminar registros")
selected_rows = st.session_state.get("tabla_seleccion", {}).get("selection", {}).get("rows", [])
with col2:
    if len(selected_rows) > 0:
        idx = selected_rows[0]
        id_sel = dt_emp.iloc[idx]["id"]
        st.write("ID: ", id_sel)
        crud_empleado.obtener_registro(id_sel)
        if st.button("Modif Salario"):
            crud_empleado.actualizar_registro(id_sel, "salario_base", e4_salario)
            st.session_state.df_empleado4 = crud_empleado.df
            if "tabla_seleccion" in st.session_state:
                st.session_state["tabla_seleccion"]={"selection":{"rows":[],"columns": []}}
            st.rerun()
    else:
        st.warning("Selecciona una fila")

with col3:
    if len(selected_rows) > 0:
        idx = selected_rows[0]
        id_sel = dt_emp.iloc[idx]["id"]
        st.write("ID: ", id_sel)
        crud_empleado.obtener_registro(id_sel)
        if st.button("Eliminar"):
```

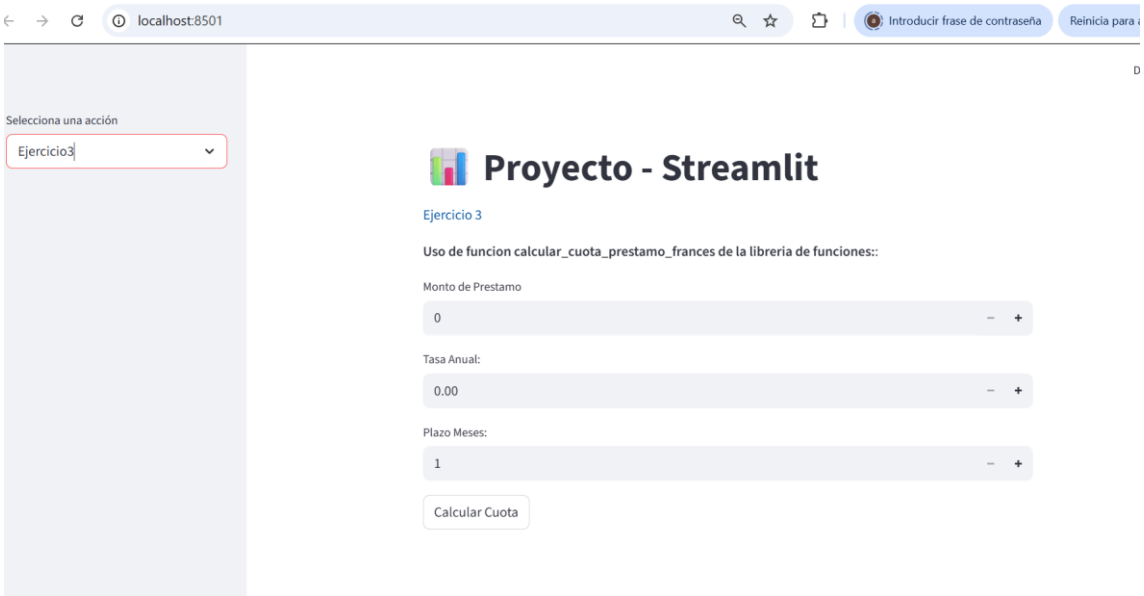
```
            crud_empleado.eliminar_registro(id_sel)
            st.session_state.df_empleado4 = crud_empleado.df
            if "tabla_seleccion" in st.session_state:
                st.session_state["tabla_seleccion"]={"selection":{"rows":[],"columns": []}}
            st.rerun()
        else:
            st.warning("Selecciona una fila")


    #with col4:
    #    if len(selected_rows) > 0:
    #        idx = selected_rows[0]
    #        id_sel = dt_emp.iloc[idx]["id"]
    #        st.write("ID: ", id_sel)
    #        if st.button("Obtener"):
    #            crud_empleado.obtener_registro(id_sel)
    #            e4_nombre_key = crud_empleado.df["nombre"]
    #            e4_salario_key = crud_empleado.df["salario_base"]
    #            st.session_state.df_empleado4 = crud_empleado.df
    #            if "tabla_seleccion" in st.session_state:
    #                st.session_state["tabla_seleccion"]={"selection":{"rows":[],"columns": []}}
    #        st.rerun()
    #    else:
    #        st.warning("Selecciona una fila")



    event = st.dataframe(dt_emp, width='stretch',on_select="rerun", selection_mode="single-row", key="tabla_seleccion") # st.dataframe se ve más ordenado que st.write
    st.write("longitud de len2 : ", len(dt_emp))
```

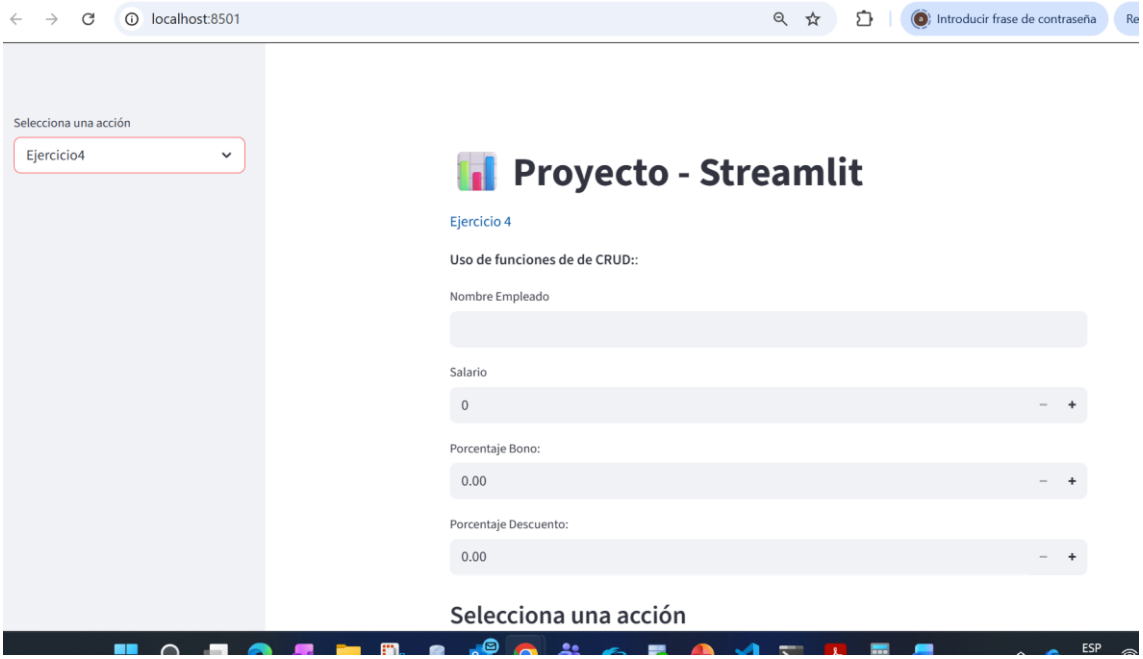
Pantalla1:


localhost:8501
Introducir frase de contraseña
Selecciona una acción
Home
Proyecto - Streamlit
Módulo 1 – Python Fundamentals
Nombre: Alex Arturo Ortega Soto
2026
Este proyecto busca mostrar los conceptos aprendidos durante el primer modulo de Python Fundamentals
Tecnologias usadas:
Python: (Pandas, Numpy, Streamlit)
Git


localhost:8501
Introducir frase de contraseña
Selecciona una acción
Ejercicio1
Proyecto - Streamlit
Ejercicio 1
Aplicar y demostrar el uso de dataframe y captura de datos usando controles como::
Selectbox
number_input
text_input
Concepto
Tipo Movimiento
Ingreso
Ingreso Real
0.00


localhost:8501
Introducir frase de contraseña
Reinicia par
Selecciona una acción
Ejercicio2
Proyecto - Streamlit
Ejercicio 2
Aplicar y demostrar el uso de librerias como NumPy y manejo de Arrays y Dataframe::
Nombre de Producto
Categoria
Alimentos
Precio:
0.00
Cantidad:
0


localhost:8501
Introducir frase de contraseña
Reinicia para
Selecciona una acción
Ejercicio3
Proyecto - Streamlit
Ejercicio 3
Uso de funcion calcular_cuota_prestamo_frances de la libreria de funciones::
Monto de Prestamo
0
Tasa Anual:
0.00
Plazo Meses:
1
Calcular Cuota


localhost:8501
Introducir frase de contraseña
Selecciona una acción
Ejercicio4
Proyecto - Streamlit
Ejercicio 4
Uso de funciones de de CRUD::
Nombre Empleado
Salario
0
Porcentaje Bono:
0.00
Porcentaje Descuento:
0.00
Selecciona una acción
ESP